simplificationOut

April 28, 2026

[1]:
```
import warnings
warnings.filterwarnings("ignore")

from pathlib import Path
import subprocess
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    roc_auc_score,
    average_precision_score,
    brier_score_loss,
 )
from sklearn.calibration import CalibratedClassifierCV, calibration_curve
from sklearn.isotonic import IsotonicRegression

try:
    from lifelines import KaplanMeierFitter, CoxPHFitter
    from lifelines.utils import concordance_index
except ModuleNotFoundError:
    print("Installing lifelines...")
    subprocess.check_call([sys.executable, "-m", "pip", "install", "lifelines",␣
 ↪"-q"])

    from lifelines import KaplanMeierFitter, CoxPHFitter
    from lifelines.utils import concordance_index
```

[2]:
```
RANDOM_STATE = 42
```

```
USE_SAMPLE = False
SAMPLE_SIZE = 250_000

SIX_HORIZON = 6
SHORT_HORIZON = 12
EIGHTEEN_HORIZON = 18
LONG_HORIZON = 24

TRAIN_END = "2014-12-31"
VALID_START, VALID_END = "2015-01-01", "2015-12-31"
TEST_START, TEST_END = "2016-01-01", "2019-12-31"

DATA_DIR = Path("data")
RAW_DIR = DATA_DIR / "raw"
OUTPUT_DIR = DATA_DIR / "simplification_PD"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

print("Output directory:", OUTPUT_DIR.resolve())
```

```
Output directory:
/Users/pattonpatton/Desktop/RiskProject/Risk_Management_Project/Time
Based/data/simplification_PD
```

[3]:
```
def find_accepted_file():
    candidates = [
        Path("data/raw/lendingclub/accepted_2007_to_2018Q4.csv.gz"),
        Path("data/raw/accepted_2007_to_2018Q4.csv.gz"),
        Path("accepted_2007_to_2018Q4.csv.gz"),
        Path("/mnt/data/accepted_2007_to_2018Q4.csv.gz"),
        Path("/mnt/data/data/raw/lendingclub/accepted_2007_to_2018Q4.csv.gz"),
    ]
    for path in candidates:
        if path.exists():
            return path
    raise FileNotFoundError(
        "Could not find accepted_2007_to_2018Q4.csv.gz. Put it in data/raw/
 ↪lendingclub/ "
        "or edit find_accepted_file()."
    )

accepted_path = find_accepted_file()
print("Using raw file:", accepted_path)
```

```
Using raw file: data/raw/lendingclub/accepted_2007_to_2018Q4.csv.gz
```

[4]:
```
key_columns = [
    "id",
    "loan_status",
```

```
    "issue_d",
    "term",
    "grade",
    "int_rate",
    "loan_amnt",
    "installment",
    "emp_length",
    "home_ownership",
    "annual_inc",
    "verification_status",
    "purpose",
    "addr_state",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_range_low",
    "fico_range_high",
    "earliest_cr_line",
    "last_pymnt_d",
    "last_credit_pull_d",
]

df_raw = pd.read_csv(
    accepted_path,
    usecols=lambda c: c in key_columns,
    low_memory=False,
    compression="gzip",
)

if USE_SAMPLE:
    df_raw = df_raw.sample(min(SAMPLE_SIZE, len(df_raw)),␣
 ↪random_state=RANDOM_STATE).copy()

print("Raw shape:", df_raw.shape)
display(df_raw.head(3))
```

```
Raw shape: (2260701, 29)
```

	id	loan_amnt	term	int_rate	installment	grade	emp_length	\
0	68407277	3600.0	36 months	13.99	123.03	C	10+ years	
1	68355089	24700.0	36 months	11.99	820.28	C	10+ years	

```
2  68341763    20000.0   60 months     10.78       432.66     B  10+ years

  home_ownership  annual_inc verification_status  … inq_last_6mths open_acc  \
0       MORTGAGE     55000.0        Not Verified  …            1.0      7.0
1       MORTGAGE     65000.0        Not Verified  …            4.0     22.0
2       MORTGAGE     63000.0        Not Verified  …            0.0      6.0

  pub_rec revol_bal  revol_util  total_acc last_pymnt_d  last_credit_pull_d  \
0     0.0    2765.0        29.7       13.0     Jan-2019            Mar-2019
1     0.0   21470.0        19.2       38.0     Jun-2016            Mar-2019
2     0.0    7869.0        56.2       18.0     Jun-2017            Mar-2019

   mort_acc  pub_rec_bankruptcies
0       1.0                   0.0
1       4.0                   0.0
2       5.0                   0.0

[3 rows x 29 columns]
```

1 2. Basic Cleaning

[5]:
```
df = df_raw.copy()

for c in ["issue_d", "earliest_cr_line", "last_pymnt_d", "last_credit_pull_d"]:
    df[c] = pd.to_datetime(df[c], format="%b-%Y", errors="coerce")

status = df["loan_status"].fillna("").astype(str)

default_mask = status.str.contains("Charged Off|Default", case=False,␣
 ↪regex=True)
payoff_mask = status.str.contains("Fully Paid", case=False, regex=False) &␣
 ↪(~default_mask)

df["default_event"] = default_mask.astype(int)
df["payoff_event"] = payoff_mask.astype(int)
df["event_type"] = np.select(
    [df["default_event"] == 1, df["payoff_event"] == 1],
    [1, 2],
    default=0,
)

df["term_num"] = (
    df["term"].astype(str).str.extract(r"(\d+)", expand=False).astype(float)
)

# Strip percentage signs where needed.
```

```
df["int_rate"] = (
    df["int_rate"].astype(str).str.replace("%", "", regex=False)
)
df["int_rate"] = pd.to_numeric(df["int_rate"], errors="coerce")

df["fico_avg"] = (df["fico_range_low"] + df["fico_range_high"]) / 2
df["credit_history_years"] = (
    (df["issue_d"] - df["earliest_cr_line"]).dt.days / 365.25
)

# Origination-safe engineered features
income_denom = df["annual_inc"].replace(0, np.nan)
df["loan_to_income"] = df["loan_amnt"] / income_denom
df["installment_to_income"] = df["installment"] / income_denom
df["inq_to_total_acc"] = df["inq_last_6mths"] / (df["total_acc"].fillna(0) + 1)
df["delinq_flag"] = (df["delinq_2yrs"].fillna(0) > 0).astype(int)
df["fico_x_dti"] = df["fico_avg"] * df["dti"]
df[["loan_to_income", "installment_to_income", "inq_to_total_acc",␣
 ↪"fico_x_dti"]] = df[["loan_to_income", "installment_to_income",␣
 ↪"inq_to_total_acc", "fico_x_dti"]].replace([np.inf, -np.inf], np.nan)

df["end_date"] = df["last_pymnt_d"]
missing_end = df["end_date"].isna()
df.loc[missing_end, "end_date"] = df.loc[missing_end, "last_credit_pull_d"]

def month_diff(start, end):
    return (end.dt.year - start.dt.year) * 12 + (end.dt.month - start.dt.month)

df = df[df["issue_d"].notna()].copy()
df = df[df["end_date"].notna()].copy()

df["duration_months"] = month_diff(df["issue_d"], df["end_date"]).clip(lower=1)

numeric_cols = [
    "int_rate",
    "loan_amnt",
    "installment",
    "annual_inc",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
    "mort_acc",
```

```
    "pub_rec_bankruptcies",
    "fico_avg",
    "credit_history_years",
    "term_num",
    "loan_to_income",
    "installment_to_income",
    "inq_to_total_acc",
    "delinq_flag",
    "fico_x_dti",
]

for c in numeric_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")

df_model = df[df["issue_d"] <= pd.Timestamp(TEST_END)].copy()

print("Clean modeling shape:", df_model.shape)
print("Default share:", round(df_model["default_event"].mean(), 4))
print("Payoff share :", round(df_model["payoff_event"].mean(), 4))
display(df_model[["id", "issue_d", "loan_status", "default_event",␣
 ↪"payoff_event", "duration_months"]].head())
```

```
Clean modeling shape: (2260668, 42)
Default share: 0.1192
Payoff share : 0.4772
```

```
Clean modeling shape: (2260668, 42)
Default share: 0.1192
Payoff share : 0.4772
```

	id	issue_d	loan_status	default_event	payoff_event	duration_months
0	68407277	2015-12-01	Fully Paid	0	1	37
1	68355089	2015-12-01	Fully Paid	0	1	6
2	68341763	2015-12-01	Fully Paid	0	1	18
3	66310712	2015-12-01	Current	0	0	38
4	68476807	2015-12-01	Fully Paid	0	1	7

[6]:
```
train_loans = df_model[df_model["issue_d"] <= pd.Timestamp(TRAIN_END)].copy()
valid_loans = df_model[
    (df_model["issue_d"] >= pd.Timestamp(VALID_START)) &
    (df_model["issue_d"] <= pd.Timestamp(VALID_END))
```

```
].copy()
test_loans = df_model[
    (df_model["issue_d"] >= pd.Timestamp(TEST_START)) &
    (df_model["issue_d"] <= pd.Timestamp(TEST_END))
].copy()

split_table = pd.DataFrame([
    {"split": "train", "n_loans": len(train_loans), "issue_start":␣
 ↪train_loans["issue_d"].min(), "issue_end": train_loans["issue_d"].max()},
    {"split": "valid", "n_loans": len(valid_loans), "issue_start":␣
 ↪valid_loans["issue_d"].min(), "issue_end": valid_loans["issue_d"].max()},
    {"split": "test", "n_loans": len(test_loans), "issue_start":␣
 ↪test_loans["issue_d"].min(), "issue_end": test_loans["issue_d"].max()},
])
display(split_table)
```

	split	n_loans	issue_start	issue_end
0	train	466345	2007-06-01	2014-12-01
1	valid	421095	2015-01-01	2015-12-01
2	test	1373228	2016-01-01	2018-12-01

2 3. Build fixed-horizon labels

[7]:
```
def make_horizon_label(loans, horizon):
    out = loans.copy()
    out["known_horizon_outcome"] = (
        (out["duration_months"] >= horizon) |
        ((out["event_type"].isin([1, 2])) & (out["duration_months"] <= horizon))
    )
    out = out[out["known_horizon_outcome"]].copy()
    out[f"default_within_{horizon}m"] = (
        (out["default_event"] == 1) & (out["duration_months"] <= horizon)
    ).astype(int)
    return out

train_6 = make_horizon_label(train_loans, SIX_HORIZON)
valid_6 = make_horizon_label(valid_loans, SIX_HORIZON)
test_6 = make_horizon_label(test_loans, SIX_HORIZON)

train_12 = make_horizon_label(train_loans, SHORT_HORIZON)
valid_12 = make_horizon_label(valid_loans, SHORT_HORIZON)
test_12 = make_horizon_label(test_loans, SHORT_HORIZON)

train_18 = make_horizon_label(train_loans, EIGHTEEN_HORIZON)
valid_18 = make_horizon_label(valid_loans, EIGHTEEN_HORIZON)
test_18 = make_horizon_label(test_loans, EIGHTEEN_HORIZON)
```

```
train_24 = make_horizon_label(train_loans, LONG_HORIZON)
valid_24 = make_horizon_label(valid_loans, LONG_HORIZON)
test_24 = make_horizon_label(test_loans, LONG_HORIZON)

label_summary = pd.DataFrame([
    {"dataset": "train_6", "n": len(train_6), "default_rate":␣
 ↪train_6[f"default_within_{SIX_HORIZON}m"].mean()},
    {"dataset": "valid_6", "n": len(valid_6), "default_rate":␣
 ↪valid_6[f"default_within_{SIX_HORIZON}m"].mean()},
    {"dataset": "test_6",  "n": len(test_6),  "default_rate":␣
 ↪test_6[f"default_within_{SIX_HORIZON}m"].mean()},
    {"dataset": "train_12", "n": len(train_12), "default_rate":␣
 ↪train_12[f"default_within_{SHORT_HORIZON}m"].mean()},
    {"dataset": "valid_12", "n": len(valid_12), "default_rate":␣
 ↪valid_12[f"default_within_{SHORT_HORIZON}m"].mean()},
    {"dataset": "test_12",  "n": len(test_12),  "default_rate":␣
 ↪test_12[f"default_within_{SHORT_HORIZON}m"].mean()},
    {"dataset": "train_18", "n": len(train_18), "default_rate":␣
 ↪train_18[f"default_within_{EIGHTEEN_HORIZON}m"].mean()},
    {"dataset": "valid_18", "n": len(valid_18), "default_rate":␣
 ↪valid_18[f"default_within_{EIGHTEEN_HORIZON}m"].mean()},
    {"dataset": "test_18",  "n": len(test_18),  "default_rate":␣
 ↪test_18[f"default_within_{EIGHTEEN_HORIZON}m"].mean()},
    {"dataset": "train_24", "n": len(train_24), "default_rate":␣
 ↪train_24[f"default_within_{LONG_HORIZON}m"].mean()},
    {"dataset": "valid_24", "n": len(valid_24), "default_rate":␣
 ↪valid_24[f"default_within_{LONG_HORIZON}m"].mean()},
    {"dataset": "test_24",  "n": len(test_24),  "default_rate":␣
 ↪test_24[f"default_within_{LONG_HORIZON}m"].mean()},
])
display(label_summary)
```

	dataset	n	default_rate
0	train_6	466345	0.018220
1	valid_6	421095	0.020767
2	test_6	1247606	0.023025
3	train_12	466345	0.051022
4	valid_12	421095	0.060556
5	test_12	1016181	0.065078
6	train_18	466345	0.083374
7	valid_18	421095	0.100763
8	test_18	847087	0.109652
9	train_24	466345	0.113403
10	valid_24	421095	0.133633
11	test_24	707053	0.151501

2.1 3. Features

[8]:
```
model_numeric = [
    "loan_amnt",
    "installment",
    "annual_inc",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "total_acc",
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_avg",
    "credit_history_years",
    "term_num",
    "loan_to_income",
    "installment_to_income",
    "inq_to_total_acc",
    "delinq_flag",
    "fico_x_dti",
]

model_categorical = [
    "addr_state",
    "purpose",
    "home_ownership",
    "emp_length",
    "verification_status",
]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]), model_numeric),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore")),
        ]), model_categorical),
    ]
)
```

2.2 4. LPM, Logit, RF

[9]:
```
def fit_frequency_models(train_df, valid_df, test_df, horizon):
    ycol = f"default_within_{horizon}m"
    X_train = train_df[model_numeric + model_categorical]
    y_train = train_df[ycol]
    X_valid = valid_df[model_numeric + model_categorical]
    y_valid = valid_df[ycol]
    X_test = test_df[model_numeric + model_categorical]
    y_test = test_df[ycol]

    lpm = Pipeline([
        ("prep", preprocessor),
        ("model", LinearRegression()),
    ])
    logit = Pipeline([
        ("prep", preprocessor),
        ("model", LogisticRegression(max_iter=2000, C=0.5,␣
 ↪random_state=RANDOM_STATE)),
    ])
    rf_raw = Pipeline([
        ("prep", preprocessor),
        ("model", RandomForestClassifier(
            n_estimators=200,
            max_depth=12,
            min_samples_leaf=50,
            max_features="sqrt",
            class_weight="balanced_subsample",
            max_samples=0.7,
            random_state=RANDOM_STATE,
            n_jobs=-1,
        )),
    ])
    rf_raw.fit(X_train, y_train)
    rf = CalibratedClassifierCV(estimator=rf_raw, method="sigmoid", cv=3)

    lpm.fit(X_train, y_train)
    logit.fit(X_train, y_train)
    rf.fit(X_train, y_train)

    lpm_valid_raw = lpm.predict(X_valid)
    lpm_test_raw = lpm.predict(X_test)
    lpm_valid = np.clip(lpm_valid_raw, 0, 1)
    lpm_test = np.clip(lpm_test_raw, 0, 1)

    logit_valid = logit.predict_proba(X_valid)[:, 1]
    logit_test = logit.predict_proba(X_test)[:, 1]
```

```
rf_valid = rf.predict_proba(X_valid)[:, 1]
rf_test = rf.predict_proba(X_test)[:, 1]

results = pd.DataFrame([
    {
        "model": f"LPM_{horizon}m",
        "valid_roc_auc": roc_auc_score(y_valid, lpm_valid),
        "valid_pr_auc": average_precision_score(y_valid, lpm_valid),
        "valid_brier": brier_score_loss(y_valid, lpm_valid),
        "test_roc_auc": roc_auc_score(y_test, lpm_test),
        "test_pr_auc": average_precision_score(y_test, lpm_test),
        "test_brier": brier_score_loss(y_test, lpm_test),
    },
    {
        "model": f"Logit_{horizon}m",
        "valid_roc_auc": roc_auc_score(y_valid, logit_valid),
        "valid_pr_auc": average_precision_score(y_valid, logit_valid),
        "valid_brier": brier_score_loss(y_valid, logit_valid),
        "test_roc_auc": roc_auc_score(y_test, logit_test),
        "test_pr_auc": average_precision_score(y_test, logit_test),
        "test_brier": brier_score_loss(y_test, logit_test),
    },
    {
        "model": f"RF_{horizon}m",
        "valid_roc_auc": roc_auc_score(y_valid, rf_valid),
        "valid_pr_auc": average_precision_score(y_valid, rf_valid),
        "valid_brier": brier_score_loss(y_valid, rf_valid),
        "test_roc_auc": roc_auc_score(y_test, rf_test),
        "test_pr_auc": average_precision_score(y_test, rf_test),
        "test_brier": brier_score_loss(y_test, rf_test),
    },
])

payload = {
    "lpm": lpm,
    "logit": logit,
    "rf": rf,
    "rf_raw": rf_raw,
    "valid_pred_lpm": lpm_valid,
    "test_pred_lpm": lpm_test,
    "valid_pred_logit": logit_valid,
    "test_pred_logit": logit_test,
    "valid_pred_rf": rf_valid,
    "test_pred_rf": rf_test,
    "y_valid": y_valid,
    "y_test": y_test,
```

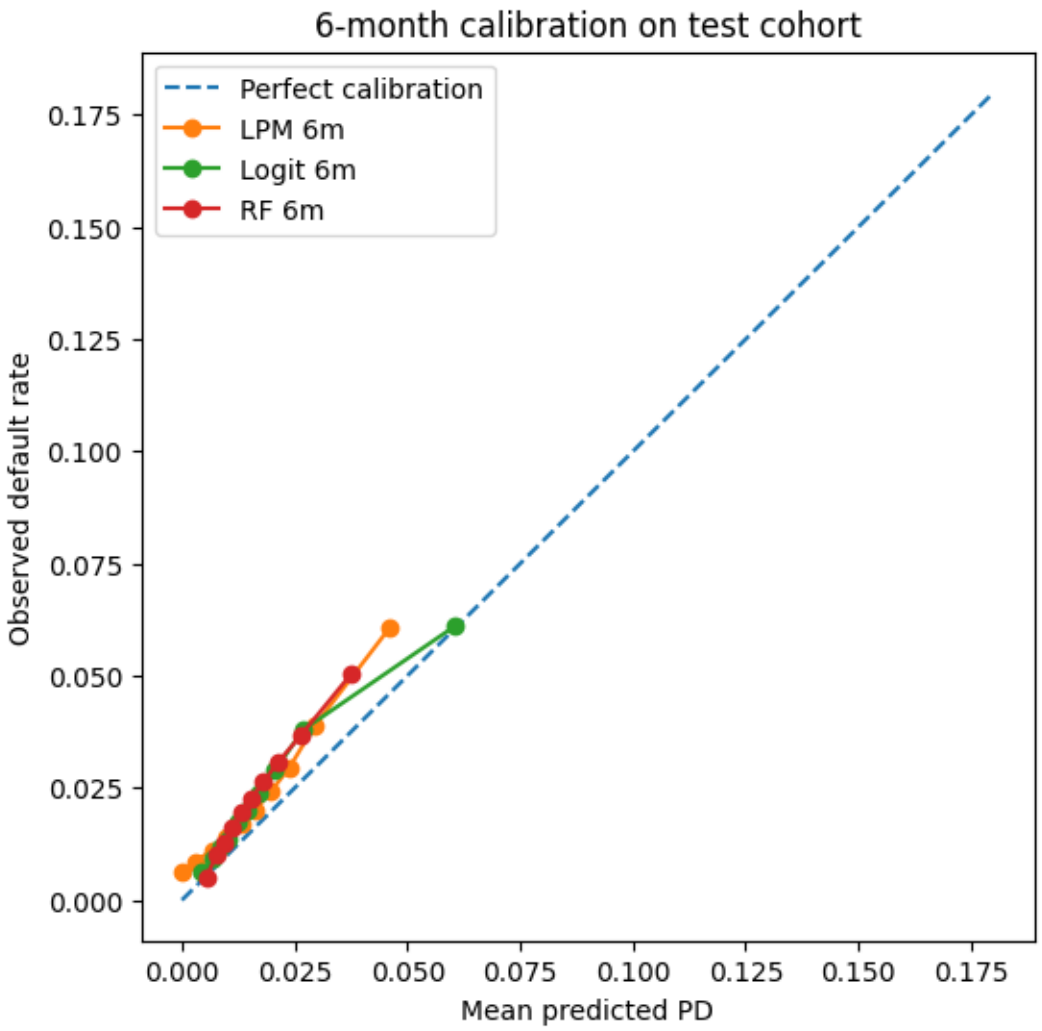
```
        "results": results,
    }
    return payload

freq6 = fit_frequency_models(train_6, valid_6, test_6, SIX_HORIZON)
freq12 = fit_frequency_models(train_12, valid_12, test_12, SHORT_HORIZON)
freq18 = fit_frequency_models(train_18, valid_18, test_18, EIGHTEEN_HORIZON)
freq24 = fit_frequency_models(train_24, valid_24, test_24, LONG_HORIZON)

frequency_results = pd.concat(
    [
        freq6["results"].assign(horizon_months=SIX_HORIZON),
        freq12["results"].assign(horizon_months=SHORT_HORIZON),
        freq18["results"].assign(horizon_months=EIGHTEEN_HORIZON),
        freq24["results"].assign(horizon_months=LONG_HORIZON),
    ],
    ignore_index=True,
)

display(frequency_results)
```

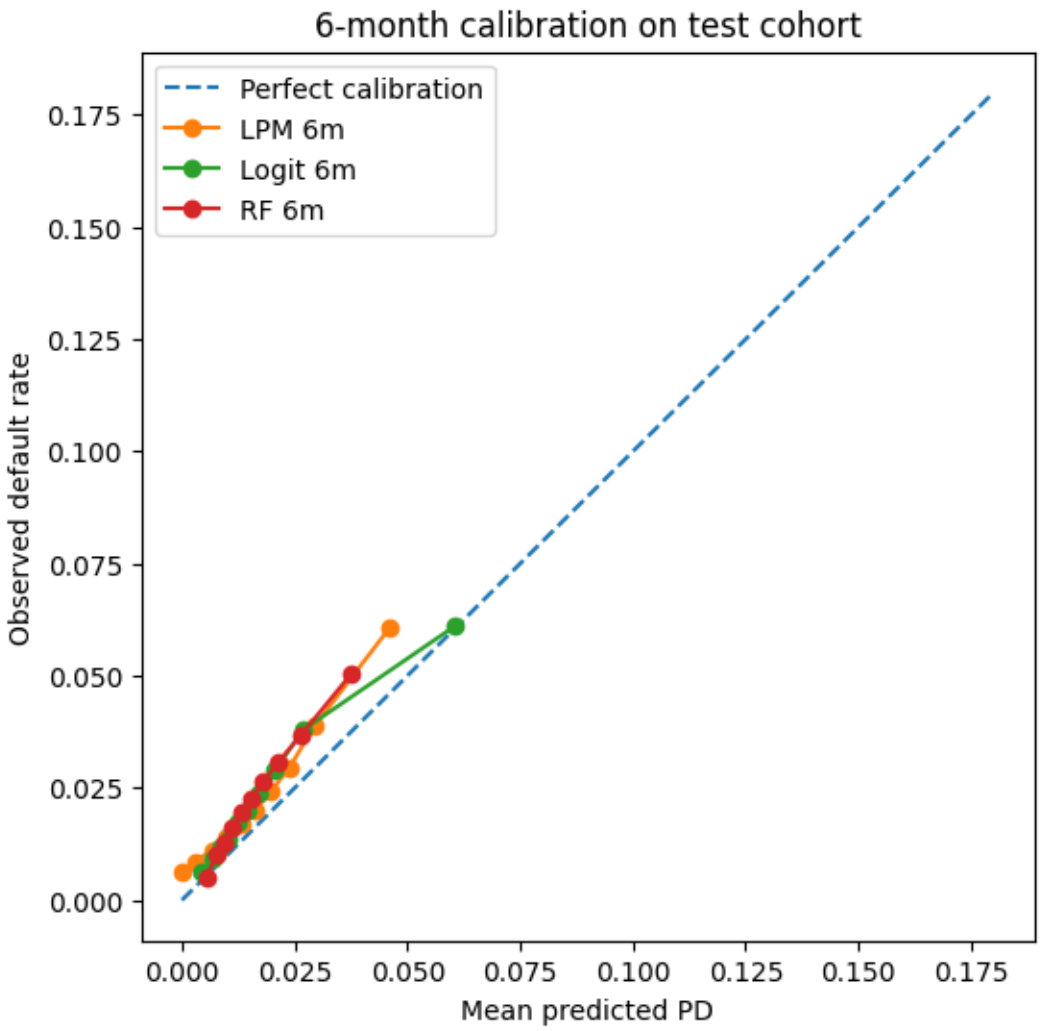
	model	valid_roc_auc	valid_pr_auc	valid_brier	test_roc_auc	test_pr_auc	test_brier	horizon_months
0	LPM_6m	0.698025	0.043929	0.020149	0.686291	0.047357	0.022415	6
1	Logit_6m	0.697541	0.046132	0.020145	0.684235	0.048753	0.023031	6
2	RF_6m	0.672954	0.039203	0.020192	0.662875	0.042169	0.022374	6
3	LPM_12m	0.691000	0.122241	0.055564	0.680430	0.125413	0.059860	12
4	Logit_12m	0.689929	0.122790	0.055531	0.677830	0.125274	0.060490	12
5	RF_12m	0.673264	0.111603	0.055767	0.663990	0.116345	0.059938	12
6	LPM_18m	0.683641	0.185799	0.087621	0.677735	0.197494	0.095094	18
7	Logit_18m	0.682818	0.187610	0.087569	0.675847	0.198873	0.095529	18
8	RF_18m	0.670218	0.176071	0.087967	0.666832	0.190046	0.095451	18
9	LPM_24m	0.681134	0.234117	0.110959	0.680005	0.261506	0.124162	24
10	Logit_24m	0.680637	0.238374	0.110807	0.679218	0.266726	0.124262	24
11	RF_24m	0.669231	0.226967	0.111383	0.671190	0.256947	0.124861	24

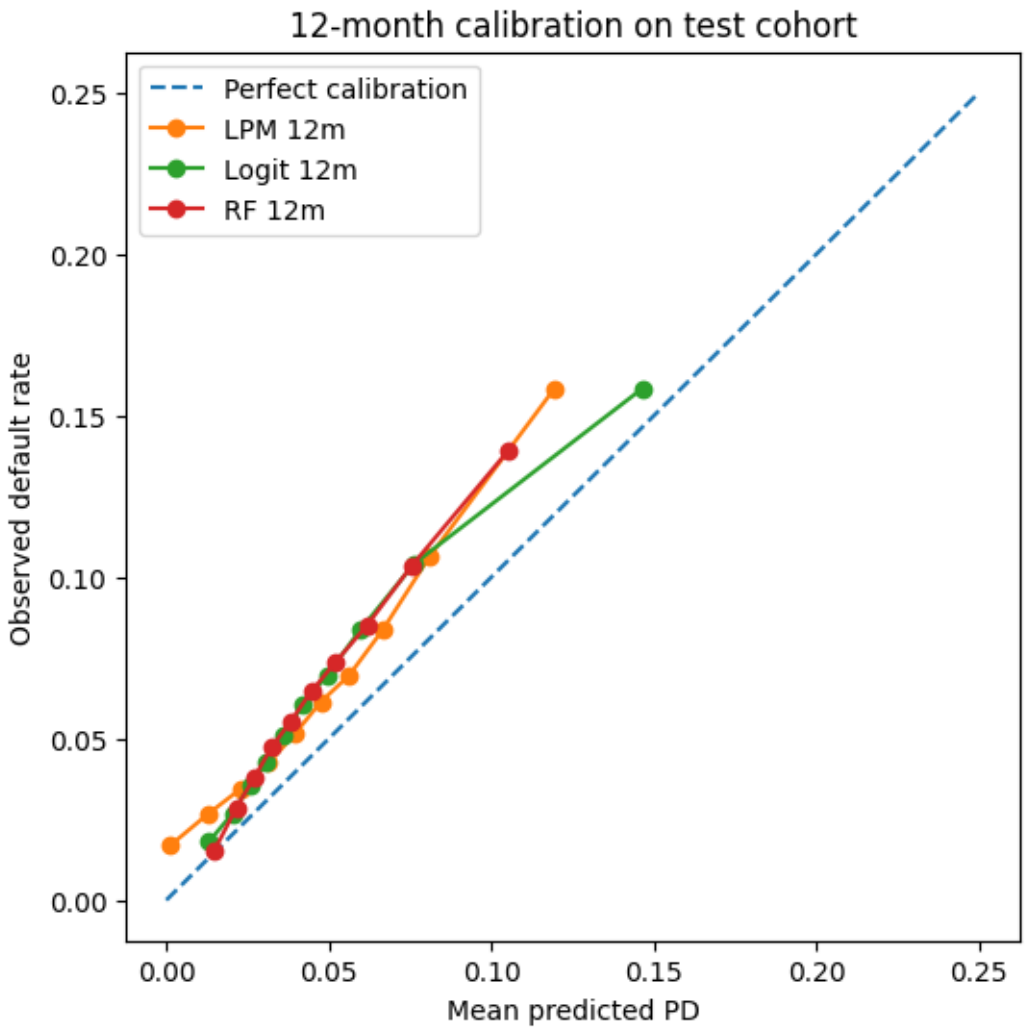
2.2.1 4.1 Accuracy visuals for the frequency models

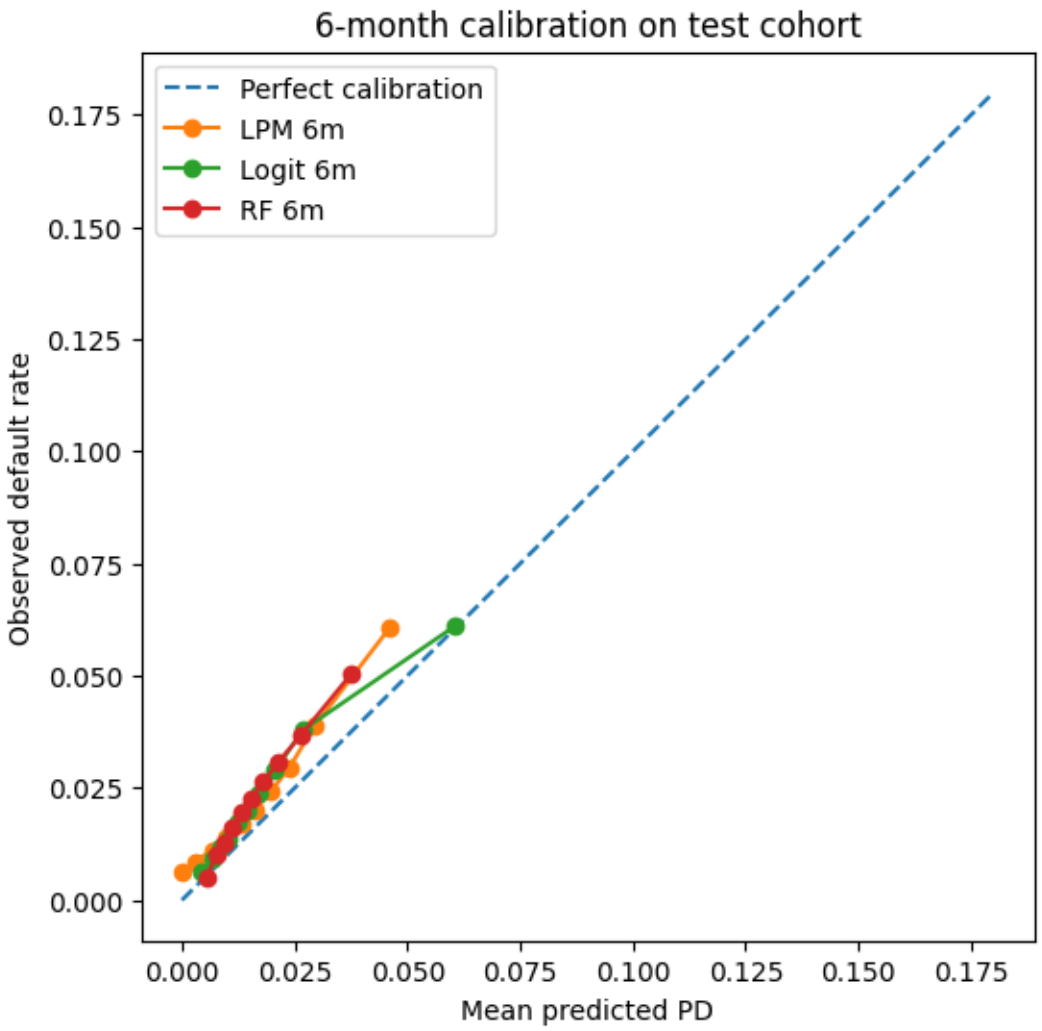
[10]:
```
# 6m calibration: LPM vs Logit vs RF
prob_true_lpm_6, prob_pred_lpm_6 = calibration_curve(freq6["y_test"],␣
 ↪freq6["test_pred_lpm"], n_bins=10, strategy="quantile")
prob_true_logit_6, prob_pred_logit_6 = calibration_curve(freq6["y_test"],␣
 ↪freq6["test_pred_logit"], n_bins=10, strategy="quantile")
prob_true_rf_6, prob_pred_rf_6 = calibration_curve(freq6["y_test"],␣
 ↪freq6["test_pred_rf"], n_bins=10, strategy="quantile")

plt.figure(figsize=(6, 6))
plt.plot([0, 0.18], [0, 0.18], linestyle="--", label="Perfect calibration")
plt.plot(prob_pred_lpm_6, prob_true_lpm_6, marker="o", label="LPM 6m")
plt.plot(prob_pred_logit_6, prob_true_logit_6, marker="o", label="Logit 6m")
plt.plot(prob_pred_rf_6, prob_true_rf_6, marker="o", label="RF 6m")
plt.xlabel("Mean predicted PD")
plt.ylabel("Observed default rate")
plt.title("6-month calibration on test cohort")
plt.legend()
plt.show()

# 12m calibration: LPM vs Logit vs RF
prob_true_lpm_12, prob_pred_lpm_12 = calibration_curve(freq12["y_test"],␣
 ↪freq12["test_pred_lpm"], n_bins=10, strategy="quantile")
prob_true_logit_12, prob_pred_logit_12 = calibration_curve(freq12["y_test"],␣
 ↪freq12["test_pred_logit"], n_bins=10, strategy="quantile")
prob_true_rf_12, prob_pred_rf_12 = calibration_curve(freq12["y_test"],␣
 ↪freq12["test_pred_rf"], n_bins=10, strategy="quantile")

plt.figure(figsize=(6, 6))
plt.plot([0, 0.25], [0, 0.25], linestyle="--", label="Perfect calibration")
plt.plot(prob_pred_lpm_12, prob_true_lpm_12, marker="o", label="LPM 12m")
plt.plot(prob_pred_logit_12, prob_true_logit_12, marker="o", label="Logit 12m")
plt.plot(prob_pred_rf_12, prob_true_rf_12, marker="o", label="RF 12m")
plt.xlabel("Mean predicted PD")
plt.ylabel("Observed default rate")
plt.title("12-month calibration on test cohort")
plt.legend()
plt.show()

# 18m calibration: LPM vs Logit vs RF
prob_true_lpm_18, prob_pred_lpm_18 = calibration_curve(freq18["y_test"],␣
 ↪freq18["test_pred_lpm"], n_bins=10, strategy="quantile")
prob_true_logit_18, prob_pred_logit_18 = calibration_curve(freq18["y_test"],␣
 ↪freq18["test_pred_logit"], n_bins=10, strategy="quantile")
prob_true_rf_18, prob_pred_rf_18 = calibration_curve(freq18["y_test"],␣
 ↪freq18["test_pred_rf"], n_bins=10, strategy="quantile")
```

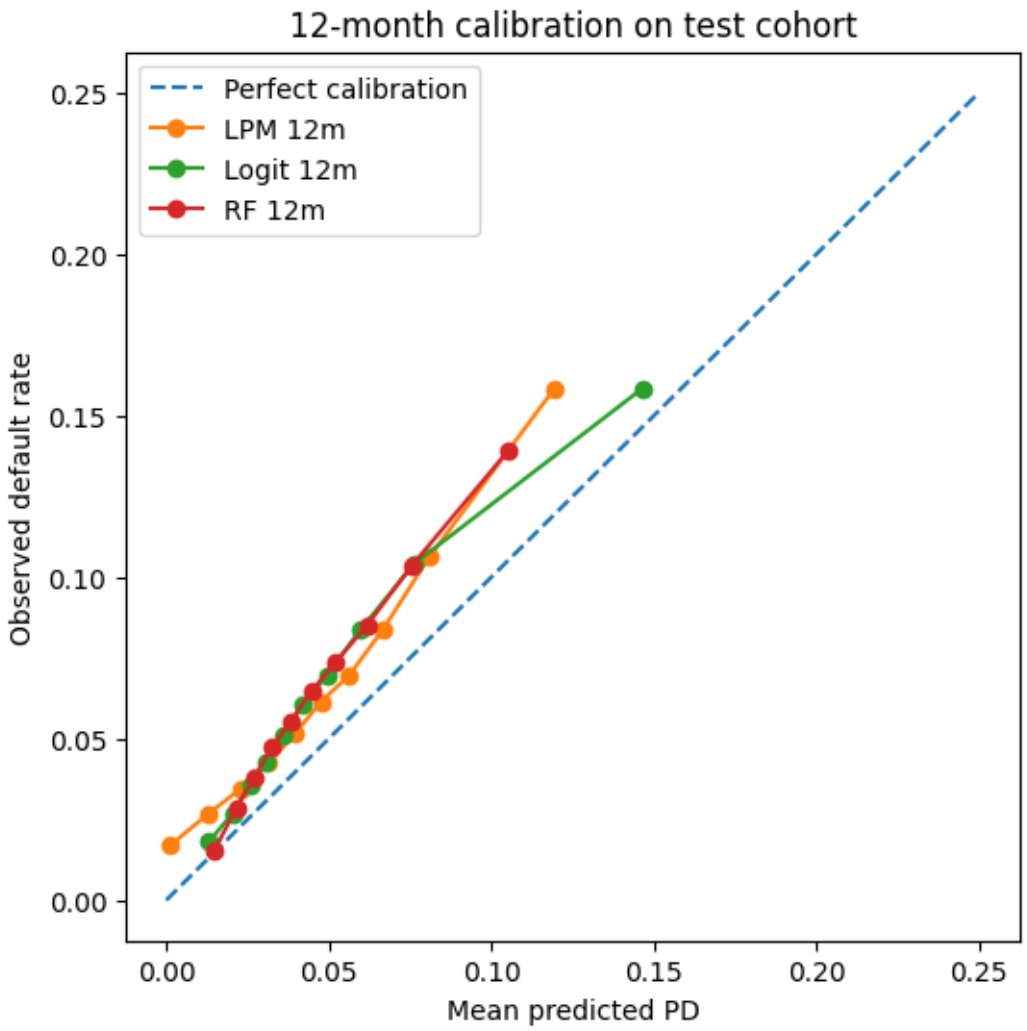
```
plt.figure(figsize=(6, 6))
plt.plot([0, 0.32], [0, 0.32], linestyle="--", label="Perfect calibration")
plt.plot(prob_pred_lpm_18, prob_true_lpm_18, marker="o", label="LPM 18m")
plt.plot(prob_pred_logit_18, prob_true_logit_18, marker="o", label="Logit 18m")
plt.plot(prob_pred_rf_18, prob_true_rf_18, marker="o", label="RF 18m")
plt.xlabel("Mean predicted PD")
plt.ylabel("Observed default rate")
plt.title("18-month calibration on test cohort")
plt.legend()
plt.show()

# 24m calibration: compare all fixed-horizon models against Cox later
prob_true_logit_24, prob_pred_logit_24 = calibration_curve(freq24["y_test"],␣
 ↪freq24["test_pred_logit"], n_bins=10, strategy="quantile")
prob_true_rf_24, prob_pred_rf_24 = calibration_curve(freq24["y_test"],␣
 ↪freq24["test_pred_rf"], n_bins=10, strategy="quantile")
plt.figure(figsize=(6, 4))
plt.plot([0, 0.40], [0, 0.40], linestyle="--", label="Perfect calibration")
plt.plot(prob_pred_logit_24, prob_true_logit_24, marker="o", label="Logit 24m")
plt.plot(prob_pred_rf_24, prob_true_rf_24, marker="o", label="RF 24m")
plt.xlabel("Mean predicted PD")
plt.ylabel("Observed default rate")
plt.title("24-month calibration on test cohort")
plt.legend()
plt.show()
```

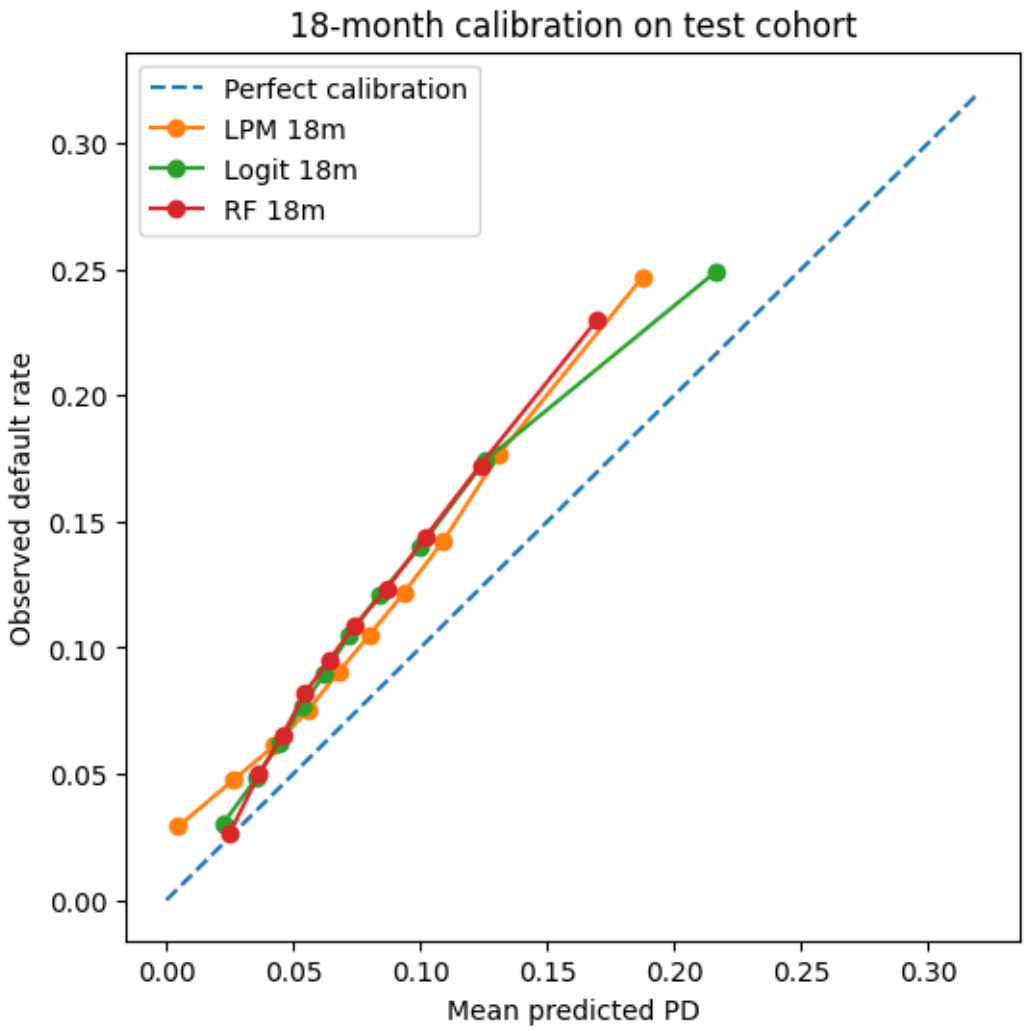

6-month calibration on test cohort
Perfect calibration
LPM 6m
Logit 6m
RF 6m
Observed default rate
Mean predicted PD
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175

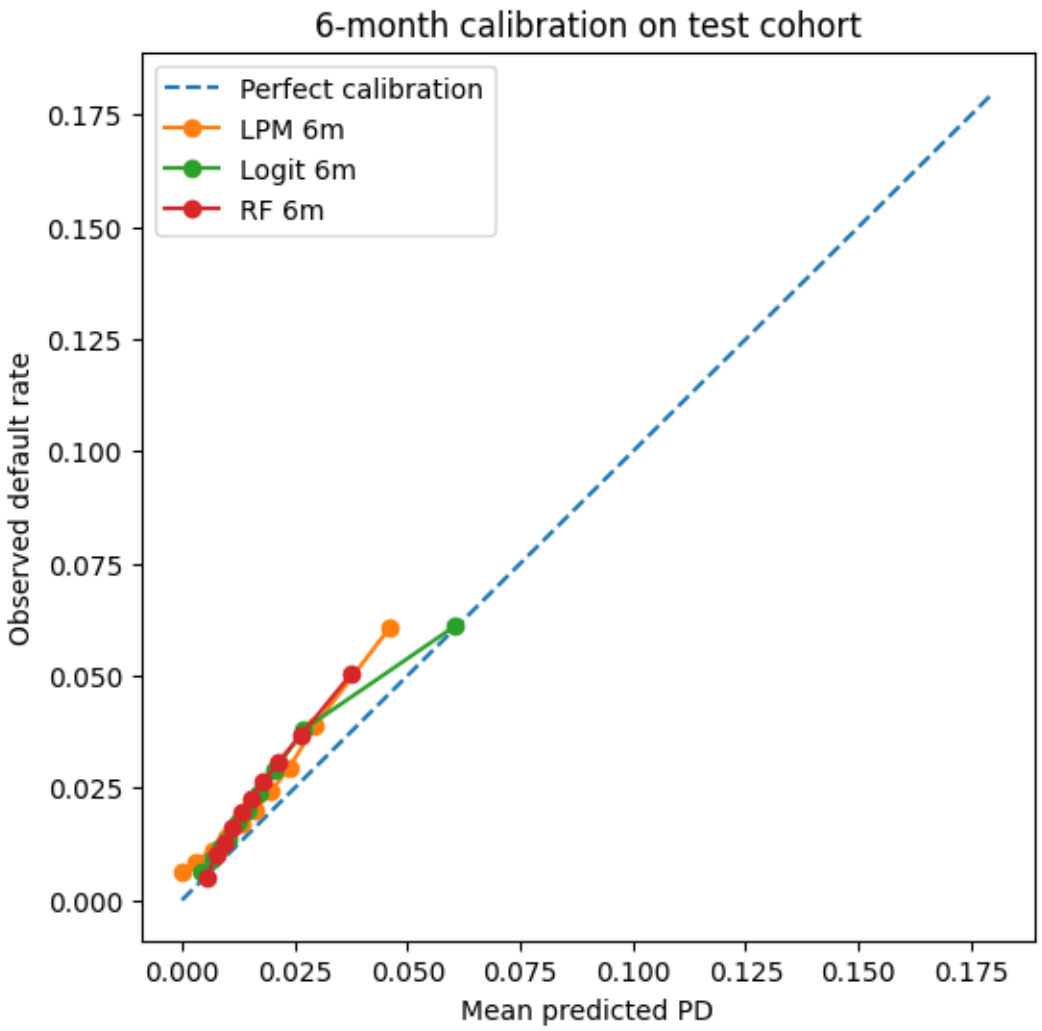

6-month calibration on test cohort
Perfect calibration
LPM 6m
Logit 6m
RF 6m
Observed default rate
Mean predicted PD
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175

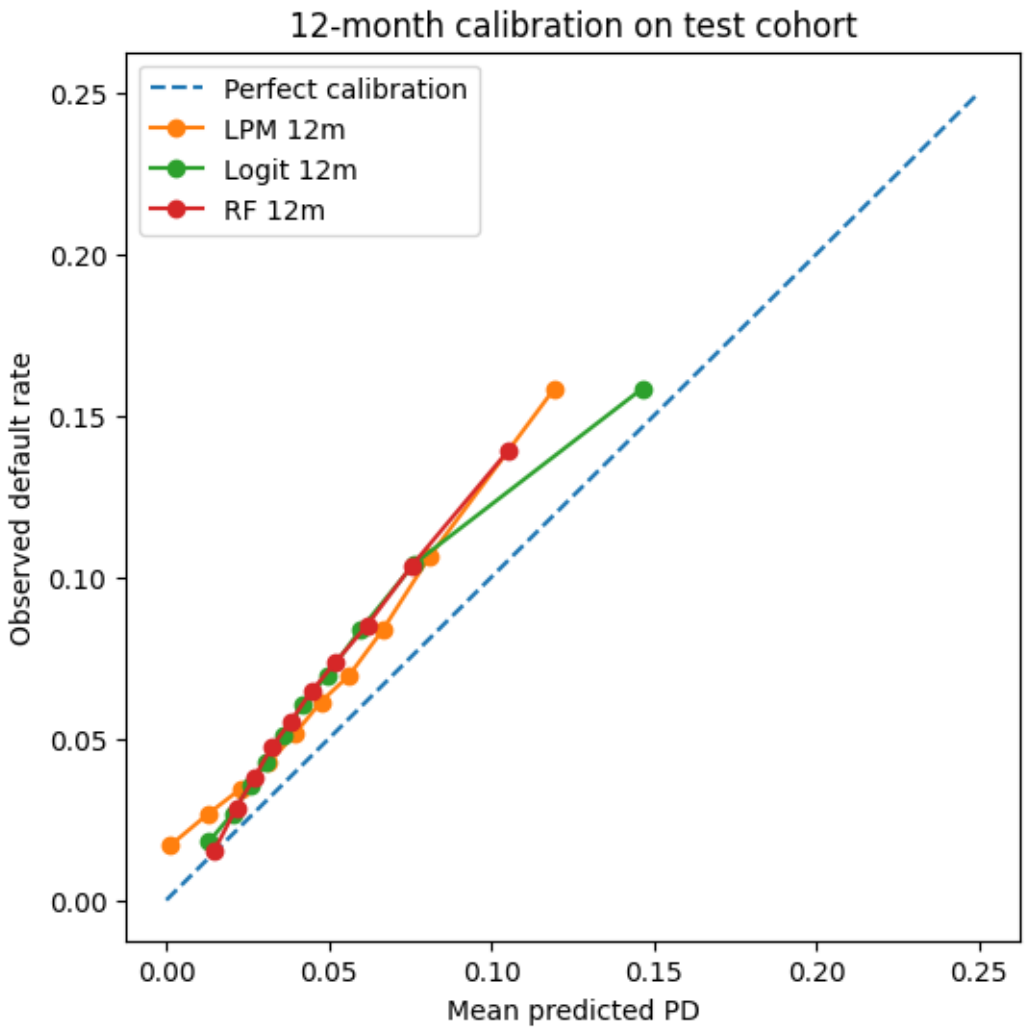

12-month calibration on test cohort
Perfect calibration
LPM 12m
Logit 12m
RF 12m
Observed default rate
Mean predicted PD
0.00
0.05
0.10
0.15
0.20
0.25

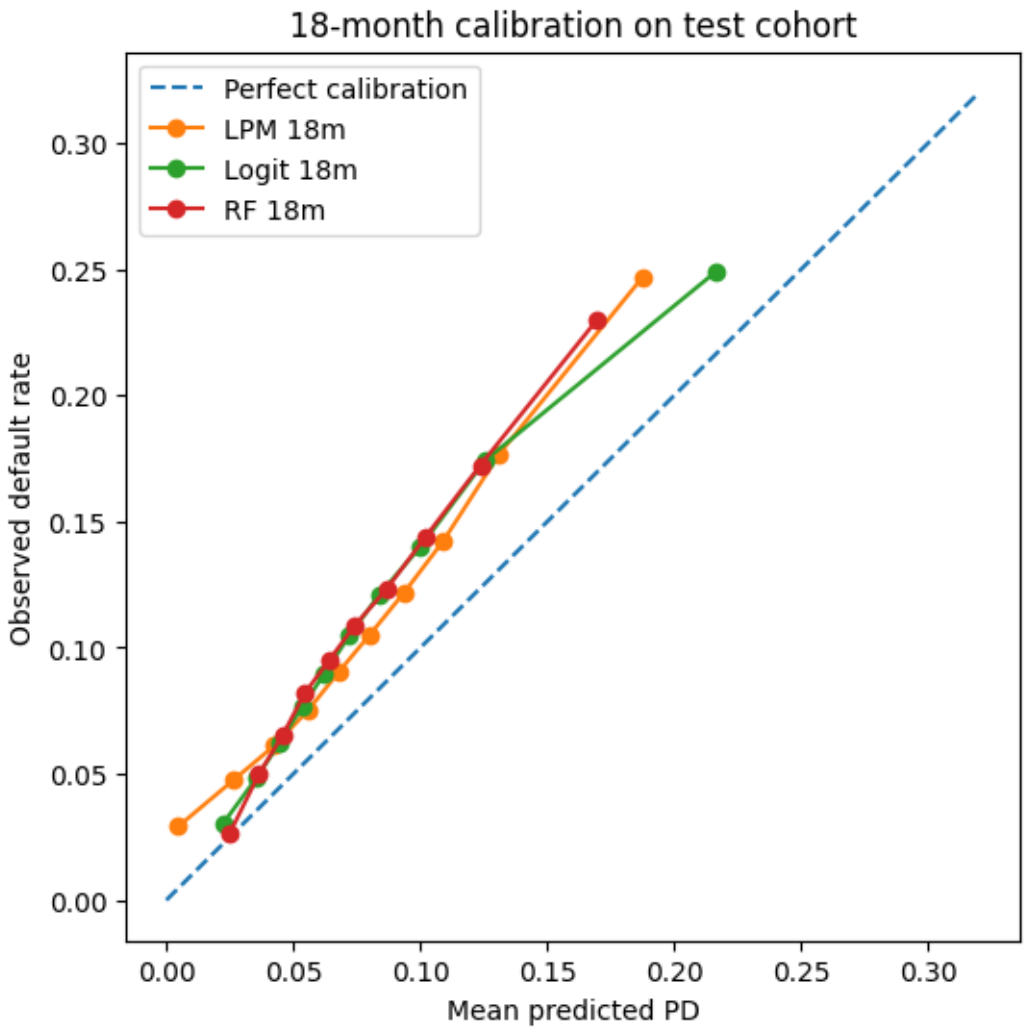

6-month calibration on test cohort
Perfect calibration
LPM 6m
Logit 6m
RF 6m
Observed default rate
Mean predicted PD
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175

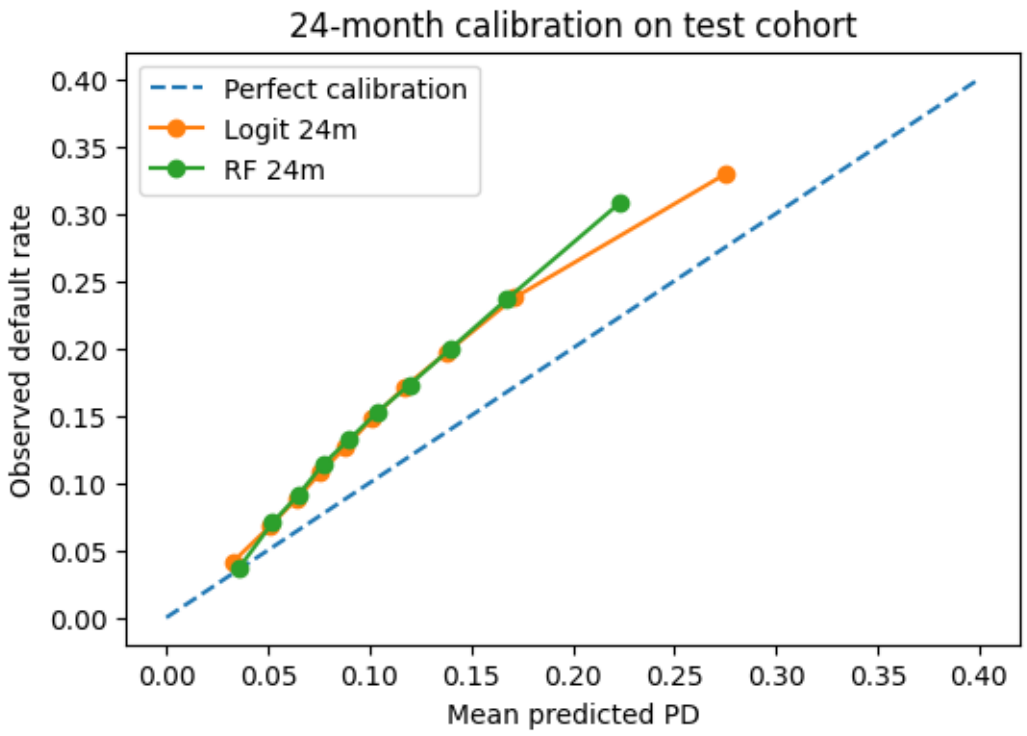

12-month calibration on test cohort
Perfect calibration
LPM 12m
Logit 12m
RF 12m
Observed default rate
Mean predicted PD
0.00
0.05
0.10
0.15
0.20
0.25


18-month calibration on test cohort
Perfect calibration
LPM 18m
Logit 18m
RF 18m
Observed default rate
Mean predicted PD
0.00
0.05
0.10
0.15
0.20
0.25
0.30


6-month calibration on test cohort
Perfect calibration
LPM 6m
Logit 6m
RF 6m
Observed default rate
Mean predicted PD
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175


12-month calibration on test cohort
Perfect calibration
LPM 12m
Logit 12m
RF 12m
Observed default rate
Mean predicted PD
0.00
0.05
0.10
0.15
0.20
0.25


18-month calibration on test cohort
Perfect calibration
LPM 18m
Logit 18m
RF 18m
Observed default rate
Mean predicted PD
0.00
0.05
0.10
0.15
0.20
0.25
0.30



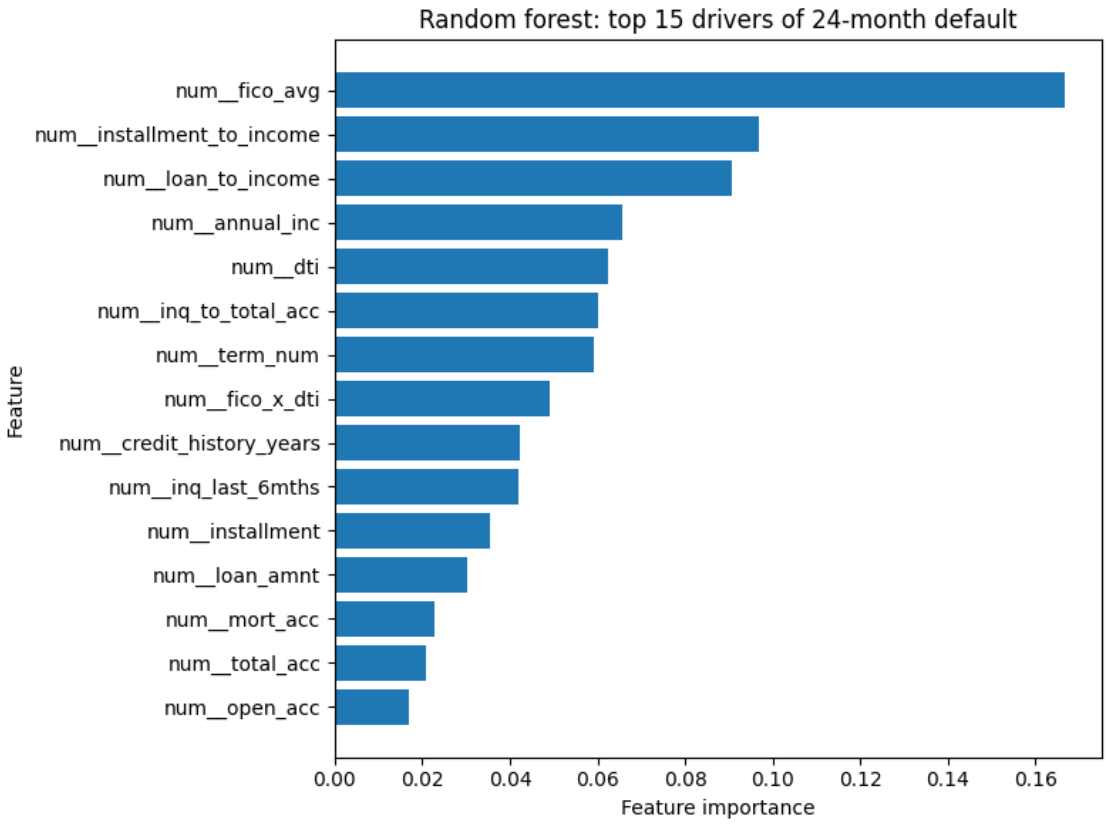
2.2.2 4.2 Random forest feature importance

[11]:
```
rf_feature_names_24 = freq24["rf_raw"].named_steps["prep"].
  ↪get_feature_names_out()
rf_feature_importance_24 = pd.DataFrame({
    "feature": rf_feature_names_24,
    "importance": freq24["rf_raw"].named_steps["model"].feature_importances_,
}).sort_values("importance", ascending=False)

display(rf_feature_importance_24.head(20))

plt.figure(figsize=(8, 6))
top_rf_importance = rf_feature_importance_24.head(15).sort_values("importance")
plt.barh(top_rf_importance["feature"], top_rf_importance["importance"])
plt.xlabel("Feature importance")
plt.ylabel("Feature")
plt.title("Random forest: top 15 drivers of 24-month default")
plt.tight_layout()
plt.show()
```

	feature	importance
11	num__fico_avg	0.166791
15	num__installment_to_income	0.096938

14	num__loan_to_income	0.090506
2	num__annual_inc	0.065743
3	num__dti	0.062365
16	num__inq_to_total_acc	0.060067
13	num__term_num	0.059054
18	num__fico_x_dti	0.049220
12	num__credit_history_years	0.042223
5	num__inq_last_6mths	0.042072
1	num__installment	0.035375
0	num__loan_amnt	0.030118
9	num__mort_acc	0.022701
8	num__total_acc	0.020739
6	num__open_acc	0.016856
70	cat__purpose_credit_card	0.016312
88	cat__home_ownership_RENT	0.015821
84	cat__home_ownership_MORTGAGE	0.015651
100	cat__verification_status_Not Verified	0.014093
80	cat__purpose_small_business	0.007908

	feature	importance
11	num__fico_avg	0.166791
15	num__installment_to_income	0.096938
14	num__loan_to_income	0.090506
2	num__annual_inc	0.065743
3	num__dti	0.062365
16	num__inq_to_total_acc	0.060067
13	num__term_num	0.059054
18	num__fico_x_dti	0.049220
12	num__credit_history_years	0.042223
5	num__inq_last_6mths	0.042072
1	num__installment	0.035375
0	num__loan_amnt	0.030118
9	num__mort_acc	0.022701
8	num__total_acc	0.020739
6	num__open_acc	0.016856
70	cat__purpose_credit_card	0.016312
88	cat__home_ownership_RENT	0.015821
84	cat__home_ownership_MORTGAGE	0.015651
100	cat__verification_status_Not Verified	0.014093
80	cat__purpose_small_business	0.007908



2.3 5. Cox PH

[12]:
```
cox_numeric = model_numeric.copy()
cox_categorical = model_categorical.copy()

def prepare_cox_design(train_df, eval_df):
    train_term = train_df[["id", "duration_months", "default_event"] +␣
 ↪cox_numeric + cox_categorical].copy()
    eval_term = eval_df[["id", "duration_months", "default_event"] +␣
 ↪cox_numeric + cox_categorical].copy()

    for frame in [train_term, eval_term]:
        for c in cox_numeric + ["duration_months", "default_event"]:
            frame[c] = pd.to_numeric(frame[c], errors="coerce")
        for c in cox_categorical:
            frame[c] = frame[c].astype("string").fillna("Missing")

    train_term = train_term.dropna(subset=["duration_months", "default_event"]).
 ↪copy()
```

```
    eval_term = eval_term.dropna(subset=["duration_months", "default_event"]).
↪copy()

    train_medians = train_term[cox_numeric].median(numeric_only=True)
    train_term[cox_numeric] = train_term[cox_numeric].fillna(train_medians)
    eval_term[cox_numeric] = eval_term[cox_numeric].fillna(train_medians)

    # Stabilize Cox optimization: cap heavy tails and standardize numeric inputs
    q_low = train_term[cox_numeric].quantile(0.01)
    q_high = train_term[cox_numeric].quantile(0.99)
    train_term[cox_numeric] = train_term[cox_numeric].clip(lower=q_low,␣
↪upper=q_high, axis=1)
    eval_term[cox_numeric] = eval_term[cox_numeric].clip(lower=q_low,␣
↪upper=q_high, axis=1)

    train_mean = train_term[cox_numeric].mean()
    train_std = train_term[cox_numeric].std().replace(0, 1)
    train_term[cox_numeric] = (train_term[cox_numeric] - train_mean) / train_std
    eval_term[cox_numeric] = (eval_term[cox_numeric] - train_mean) / train_std

    train_dm = pd.get_dummies(train_term, columns=cox_categorical,␣
↪drop_first=True)
    eval_dm = pd.get_dummies(eval_term, columns=cox_categorical,␣
↪drop_first=True)

    feature_cols = [c for c in train_dm.columns if c not in ["id",␣
↪"duration_months", "default_event"]]

    for c in feature_cols:
        if c not in eval_dm.columns:
            eval_dm[c] = 0

    extra_eval = [c for c in eval_dm.columns if c not in ["id",␣
↪"duration_months", "default_event"] + feature_cols]
    if extra_eval:
        eval_dm = eval_dm.drop(columns=extra_eval)

    train_dm = train_dm[["id", "duration_months", "default_event"] +␣
↪feature_cols].copy()
    eval_dm = eval_dm[["id", "duration_months", "default_event"] +␣
↪feature_cols].copy()

    for frame in [train_dm, eval_dm]:
        bool_cols = frame.select_dtypes(include=["bool"]).columns
        if len(bool_cols) > 0:
            frame[bool_cols] = frame[bool_cols].astype(int)
```

```
        frame[feature_cols] = frame[feature_cols].astype(float)
        frame["duration_months"] = frame["duration_months"].astype(float)
        frame["default_event"] = frame["default_event"].astype(int)

    nunique = train_dm[feature_cols].nunique(dropna=False)
    constant_cols = nunique[nunique <= 1].index.tolist()
    if constant_cols:
        feature_cols = [c for c in feature_cols if c not in constant_cols]
        train_dm = train_dm[["id", "duration_months", "default_event"] +␣
 ↪feature_cols].copy()
        eval_dm = eval_dm[["id", "duration_months", "default_event"] +␣
 ↪feature_cols].copy()

    return train_dm, eval_dm, feature_cols

cox_train, cox_valid, cox_features = prepare_cox_design(train_loans,␣
 ↪valid_loans)
_, cox_test, _ = prepare_cox_design(train_loans, test_loans)

cph = CoxPHFitter(penalizer=0.1)
cph.fit(
    cox_train.drop(columns=["id"]),
    duration_col="duration_months",
    event_col="default_event",
    show_progress=False,
)

valid_risk = cph.predict_partial_hazard(cox_valid[cox_features]).values.ravel()
test_risk = cph.predict_partial_hazard(cox_test[cox_features]).values.ravel()

cox_c_valid = concordance_index(
    cox_valid["duration_months"],
    -valid_risk,
    cox_valid["default_event"],
)
cox_c_test = concordance_index(
    cox_test["duration_months"],
    -test_risk,
    cox_test["default_event"],
)

cox_summary = pd.DataFrame([{
    "model": "Cox_PH_time_to_default",
    "valid_c_index": cox_c_valid,
    "test_c_index": cox_c_test,
    "n_train": len(cox_train),
    "n_valid": len(cox_valid),
```

```
    "n_test": len(cox_test),
}])

display(cox_summary)
```

```
                   model  valid_c_index  test_c_index  n_train  n_valid  \
0  Cox_PH_time_to_default       0.653339      0.655584   466345   421095

    n_test
0  1373228
```

2.3.1 5.1 The strongest Cox hazard ratios

[13]:
```
cox_coef_view = cph.summary[["coef", "exp(coef)", "se(coef)", "z", "p"]].
 ↪sort_values("exp(coef)", ascending=False)
display(cox_coef_view.head(20))
```

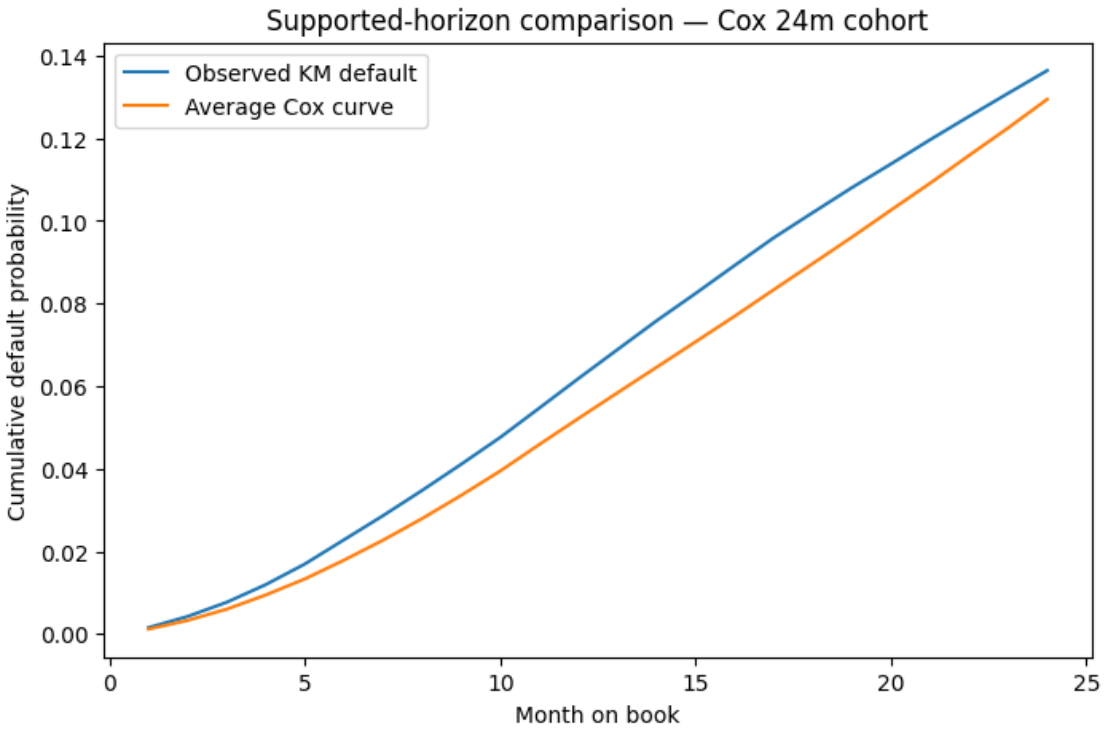
```
                              coef  exp(coef)  se(coef)          z  \
covariate
addr_state_NE             0.945249   2.573453  0.351965   2.685630
purpose_small_business    0.403724   1.497391  0.021673  18.627729
purpose_renewable_energy  0.238241   1.269014  0.103408   2.303888
purpose_moving            0.226980   1.254805  0.035241   6.440720
addr_state_IA             0.186640   1.205193  0.491914   0.379415
purpose_medical           0.184435   1.202539  0.029221   6.311812
purpose_educational       0.178093   1.194937  0.090561   1.966552
emp_length_Missing        0.154240   1.166771  0.013566  11.369866
home_ownership_OTHER      0.143452   1.154251  0.136826   1.048427
purpose_vacation          0.134939   1.144467  0.040451   3.335817
addr_state_NV             0.133345   1.142644  0.023713   5.623370
purpose_other             0.132044   1.141159  0.013462   9.808655
purpose_house             0.128802   1.137465  0.040843   3.153572
addr_state_MS             0.119665   1.127119  0.051250   2.334946
term_num                  0.107480   1.113469  0.003002  35.805712
addr_state_AL             0.089816   1.093973  0.024762   3.627153
inq_last_6mths            0.079532   1.082781  0.003383  23.508620
installment_to_income     0.075817   1.078766  0.003723  20.362257
addr_state_TN             0.071045   1.073629  0.024418   2.909516
home_ownership_RENT       0.065743   1.067952  0.007290   9.018472

                                  p
covariate
addr_state_NE              7.239312e-03
purpose_small_business     1.914785e-77
purpose_renewable_energy   2.122892e-02
purpose_moving             1.189084e-10
addr_state_IA              7.043799e-01
purpose_medical            2.757867e-10
```

```
purpose_educational         4.923489e-02
emp_length_Missing          5.907817e-30
home_ownership_OTHER        2.944421e-01
purpose_vacation            8.504905e-04
addr_state_NV               1.872673e-08
purpose_other               1.033362e-22
purpose_house               1.612854e-03
addr_state_MS               1.954622e-02
term_num                   9.000145e-281
addr_state_AL               2.865637e-04
inq_last_6mths             3.329461e-122
installment_to_income       3.615374e-92
addr_state_TN               3.619884e-03
home_ownership_RENT         1.907306e-19
```

2.3.2 5.2 Cox curve check

[14]:
```
def km_curve(loans, max_horizon=None):
    kmf = KaplanMeierFitter()
    kmf.fit(
        durations=loans["duration_months"],
        event_observed=loans["default_event"],
    )
    surv = kmf.survival_function_.reset_index()
    surv.columns = ["month_on_book", "survival"]
    surv["month_on_book"] = surv["month_on_book"].astype(float).round().
 ↪astype(int)
    surv["cum_default_km"] = 1 - surv["survival"]
    if max_horizon is not None:
        surv = surv[surv["month_on_book"] <= max_horizon].copy()
    return surv

km_test = km_curve(test_loans, max_horizon=LONG_HORIZON)

surv_pred_test = cph.predict_survival_function(cox_test[cox_features])
avg_surv = surv_pred_test.mean(axis=1).reset_index()
avg_surv.columns = ["month_on_book", "survival_cox_avg"]
avg_surv["month_on_book"] = avg_surv["month_on_book"].astype(float).round().
 ↪astype(int)
avg_surv = avg_surv.groupby("month_on_book",␣
 ↪as_index=False)["survival_cox_avg"].last()
avg_surv["cum_default_cox_avg"] = 1 - avg_surv["survival_cox_avg"]

compare_curve = km_test.merge(
    avg_surv[["month_on_book", "cum_default_cox_avg"]],
    on="month_on_book",
    how="inner",
```

```
)

plt.figure(figsize=(8, 5))
plt.plot(compare_curve["month_on_book"], compare_curve["cum_default_km"],␣
 ↪label="Observed KM default")
plt.plot(compare_curve["month_on_book"], compare_curve["cum_default_cox_avg"],␣
 ↪label="Average Cox curve")
plt.xlabel("Month on book")
plt.ylabel("Cumulative default probability")
plt.title(f"Supported-horizon comparison - Cox {LONG_HORIZON}m cohort")
plt.legend()
plt.show()
```



2.4 6. Convert Cox into a comparable PD

[15]:
```
# 18m PD from Cox on the validation/test sets
cox_valid_18 = cph.predict_survival_function(cox_valid[cox_features],␣
 ↪times=[EIGHTEEN_HORIZON]).T
cox_valid_18.columns = ["survival_18m"]
cox_valid_18["pd_18m_cox"] = 1 - cox_valid_18["survival_18m"]
cox_valid_18 = cox_valid_18.reset_index(drop=True)
cox_valid_18["id"] = cox_valid["id"].values
```

```
cox_test_18 = cph.predict_survival_function(cox_test[cox_features],␣
 ↪times=[EIGHTEEN_HORIZON]).T
cox_test_18.columns = ["survival_18m"]
cox_test_18["pd_18m_cox"] = 1 - cox_test_18["survival_18m"]
cox_test_18 = cox_test_18.reset_index(drop=True)
cox_test_18["id"] = cox_test["id"].values

valid18_compare = valid_18[["id", f"default_within_{EIGHTEEN_HORIZON}m"]].merge(
    cox_valid_18[["id", "pd_18m_cox"]],
    on="id",
    how="inner",
)
test18_compare = test_18[["id", f"default_within_{EIGHTEEN_HORIZON}m"]].merge(
    cox_test_18[["id", "pd_18m_cox"]],
    on="id",
    how="inner",
)

valid18_compare["pd_18m_logit"] = freq18["valid_pred_logit"]
test18_compare["pd_18m_logit"] = freq18["test_pred_logit"]
valid18_compare["pd_18m_rf"] = freq18["valid_pred_rf"]
test18_compare["pd_18m_rf"] = freq18["test_pred_rf"]

y_valid_18 = valid18_compare[f"default_within_{EIGHTEEN_HORIZON}m"]
y_test_18 = test18_compare[f"default_within_{EIGHTEEN_HORIZON}m"]

# 24m PD from Cox on the validation/test sets
cox_valid_24 = cph.predict_survival_function(cox_valid[cox_features],␣
 ↪times=[LONG_HORIZON]).T
cox_valid_24.columns = ["survival_24m"]
cox_valid_24["pd_24m_cox"] = 1 - cox_valid_24["survival_24m"]
cox_valid_24 = cox_valid_24.reset_index(drop=True)
cox_valid_24["id"] = cox_valid["id"].values

cox_test_24 = cph.predict_survival_function(cox_test[cox_features],␣
 ↪times=[LONG_HORIZON]).T
cox_test_24.columns = ["survival_24m"]
cox_test_24["pd_24m_cox"] = 1 - cox_test_24["survival_24m"]
cox_test_24 = cox_test_24.reset_index(drop=True)
cox_test_24["id"] = cox_test["id"].values

valid24_compare = valid_24[["id", f"default_within_{LONG_HORIZON}m"]].merge(
    cox_valid_24[["id", "pd_24m_cox"]],
    on="id",
    how="inner",
)
test24_compare = test_24[["id", f"default_within_{LONG_HORIZON}m"]].merge(
```

```
    cox_test_24[["id", "pd_24m_cox"]],
    on="id",
    how="inner",
)

valid24_compare["pd_24m_logit"] = freq24["valid_pred_logit"]
test24_compare["pd_24m_logit"] = freq24["test_pred_logit"]
valid24_compare["pd_24m_rf"] = freq24["valid_pred_rf"]
test24_compare["pd_24m_rf"] = freq24["test_pred_rf"]

y_valid_24 = valid24_compare[f"default_within_{LONG_HORIZON}m"]
y_test_24 = test24_compare[f"default_within_{LONG_HORIZON}m"]

longterm_results = pd.DataFrame([
    {
        "model": "Logit_18m",
        "valid_roc_auc": roc_auc_score(y_valid_18,␣
 ↪valid18_compare["pd_18m_logit"]),
        "valid_pr_auc": average_precision_score(y_valid_18,␣
 ↪valid18_compare["pd_18m_logit"]),
        "valid_brier": brier_score_loss(y_valid_18,␣
 ↪valid18_compare["pd_18m_logit"]),
        "test_roc_auc": roc_auc_score(y_test_18,␣
 ↪test18_compare["pd_18m_logit"]),
        "test_pr_auc": average_precision_score(y_test_18,␣
 ↪test18_compare["pd_18m_logit"]),
        "test_brier": brier_score_loss(y_test_18,␣
 ↪test18_compare["pd_18m_logit"]),
    },
    {
        "model": "Cox_implied_PD_18m",
        "valid_roc_auc": roc_auc_score(y_valid_18,␣
 ↪valid18_compare["pd_18m_cox"]),
        "valid_pr_auc": average_precision_score(y_valid_18,␣
 ↪valid18_compare["pd_18m_cox"]),
        "valid_brier": brier_score_loss(y_valid_18,␣
 ↪valid18_compare["pd_18m_cox"]),
        "test_roc_auc": roc_auc_score(y_test_18, test18_compare["pd_18m_cox"]),
        "test_pr_auc": average_precision_score(y_test_18,␣
 ↪test18_compare["pd_18m_cox"]),
        "test_brier": brier_score_loss(y_test_18, test18_compare["pd_18m_cox"]),
    },
    {
        "model": "RF_18m",
        "valid_roc_auc": roc_auc_score(y_valid_18,␣
 ↪valid18_compare["pd_18m_rf"]),
```

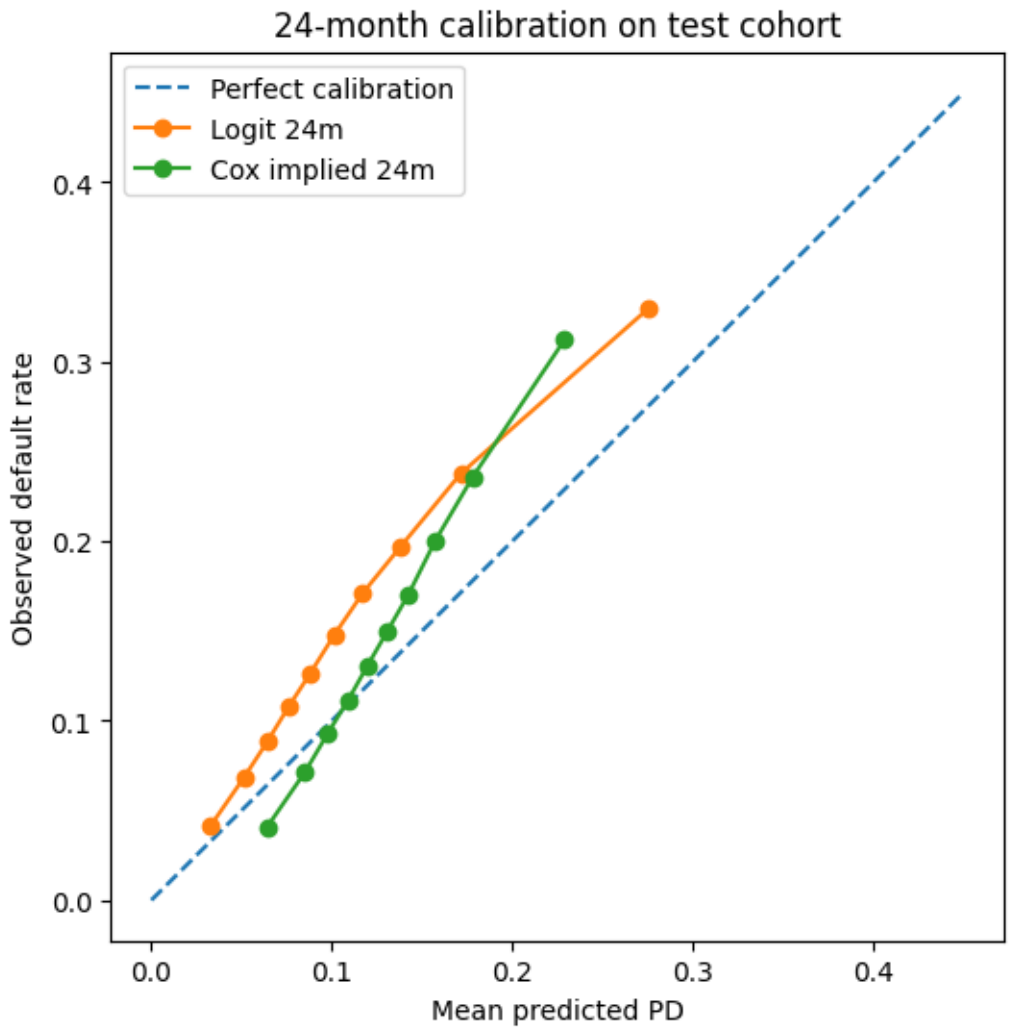
```
        "valid_pr_auc": average_precision_score(y_valid_18,␣
↪valid18_compare["pd_18m_rf"]),
        "valid_brier": brier_score_loss(y_valid_18,␣
↪valid18_compare["pd_18m_rf"]),
        "test_roc_auc": roc_auc_score(y_test_18, test18_compare["pd_18m_rf"]),
        "test_pr_auc": average_precision_score(y_test_18,␣
↪test18_compare["pd_18m_rf"]),
        "test_brier": brier_score_loss(y_test_18, test18_compare["pd_18m_rf"]),
    },
    {
        "model": "Logit_24m",
        "valid_roc_auc": roc_auc_score(y_valid_24,␣
↪valid24_compare["pd_24m_logit"]),
        "valid_pr_auc": average_precision_score(y_valid_24,␣
↪valid24_compare["pd_24m_logit"]),
        "valid_brier": brier_score_loss(y_valid_24,␣
↪valid24_compare["pd_24m_logit"]),
        "test_roc_auc": roc_auc_score(y_test_24,␣
↪test24_compare["pd_24m_logit"]),
        "test_pr_auc": average_precision_score(y_test_24,␣
↪test24_compare["pd_24m_logit"]),
        "test_brier": brier_score_loss(y_test_24,␣
↪test24_compare["pd_24m_logit"]),
    },
    {
        "model": "Cox_implied_PD_24m",
        "valid_roc_auc": roc_auc_score(y_valid_24,␣
↪valid24_compare["pd_24m_cox"]),
        "valid_pr_auc": average_precision_score(y_valid_24,␣
↪valid24_compare["pd_24m_cox"]),
        "valid_brier": brier_score_loss(y_valid_24,␣
↪valid24_compare["pd_24m_cox"]),
        "test_roc_auc": roc_auc_score(y_test_24, test24_compare["pd_24m_cox"]),
        "test_pr_auc": average_precision_score(y_test_24,␣
↪test24_compare["pd_24m_cox"]),
        "test_brier": brier_score_loss(y_test_24, test24_compare["pd_24m_cox"]),
    },
    {
        "model": "RF_24m",
        "valid_roc_auc": roc_auc_score(y_valid_24,␣
↪valid24_compare["pd_24m_rf"]),
        "valid_pr_auc": average_precision_score(y_valid_24,␣
↪valid24_compare["pd_24m_rf"]),
        "valid_brier": brier_score_loss(y_valid_24,␣
↪valid24_compare["pd_24m_rf"]),
        "test_roc_auc": roc_auc_score(y_test_24, test24_compare["pd_24m_rf"]),
```

```
        "test_pr_auc": average_precision_score(y_test_24,␣
  ↪test24_compare["pd_24m_rf"]),
        "test_brier": brier_score_loss(y_test_24, test24_compare["pd_24m_rf"]),
    },
])

display(longterm_results)
```

```
                model  valid_roc_auc  valid_pr_auc  valid_brier  test_roc_auc  \
0          Logit_18m       0.682818      0.187610     0.087569      0.675847
1  Cox_implied_PD_18m       0.670660      0.175989     0.088061      0.666398
2             RF_18m       0.670218      0.176071     0.087967      0.666832
3          Logit_24m       0.680637      0.238374     0.110807      0.679218
4  Cox_implied_PD_24m       0.670445      0.226868     0.111403      0.670676
5             RF_24m       0.669231      0.226967     0.111383      0.671190

   test_pr_auc  test_brier
0     0.198873    0.095529
1     0.188775    0.095234
2     0.190046    0.095451
3     0.266726    0.124262
4     0.254872    0.123877
5     0.256947    0.124861
```

2.4.1 6.1 Calibration: 24m logit vs 24m Cox

[16]:
```
prob_true_logit_24, prob_pred_logit_24 = calibration_curve(
    y_test_24, test24_compare["pd_24m_logit"], n_bins=10, strategy="quantile"
)
prob_true_cox_24, prob_pred_cox_24 = calibration_curve(
    y_test_24, test24_compare["pd_24m_cox"], n_bins=10, strategy="quantile"
)

plt.figure(figsize=(6, 6))
plt.plot([0, 0.45], [0, 0.45], linestyle="--", label="Perfect calibration")
plt.plot(prob_pred_logit_24, prob_true_logit_24, marker="o", label="Logit 24m")
plt.plot(prob_pred_cox_24, prob_true_cox_24, marker="o", label="Cox implied␣
  ↪24m")
plt.xlabel("Mean predicted PD")
plt.ylabel("Observed default rate")
plt.title("24-month calibration on test cohort")
plt.legend()
plt.show()
```



2.4.2 Cox Neatening

[17]:
```
# Calibrate Cox PDs at 24m, 36m, and 60m using isotonic regression.
calibration_horizons = [LONG_HORIZON, 36, 60]

if all(name in globals() for name in ["final_cph", "final_cox_all",␣
 ↪"final_cox_features"]):
    cox_model_for_cal = final_cph
    cox_design_for_cal = final_cox_all
    cox_features_for_cal = final_cox_features
else:
    cox_model_for_cal = cph
    cox_design_for_cal = pd.concat([cox_valid, cox_test], ignore_index=True)
    cox_features_for_cal = cox_features
```

```
def predict_cox_pd_for_ids(horizon, id_series):
    subset = cox_design_for_cal.loc[cox_design_for_cal["id"].isin(id_series),␣
 ↪["id"] + cox_features_for_cal].copy()
    pd_raw = 1 - cox_model_for_cal.predict_survival_function(
        subset[cox_features_for_cal],
        times=[horizon],
    ).T.iloc[:, 0].values
    return pd.DataFrame({"id": subset["id"].values, "pd_raw": pd_raw})


cox_isotonic_models = {}
cox_calibration_rows = []

for h in calibration_horizons:
    valid_h = make_horizon_label(valid_loans, h)
    test_h = make_horizon_label(test_loans, h)

    # Only calibrate horizon h on contracts that can actually run that long.
    valid_h = valid_h.loc[valid_h["term_num"] >= h, ["id",␣
 ↪f"default_within_{h}m"]].copy()
    test_h = test_h.loc[test_h["term_num"] >= h, ["id",␣
 ↪f"default_within_{h}m"]].copy()

    pred_valid_h = predict_cox_pd_for_ids(h, valid_h["id"])
    pred_test_h = predict_cox_pd_for_ids(h, test_h["id"])

    valid_cal_h = valid_h.merge(pred_valid_h, on="id", how="inner")
    test_cal_h = test_h.merge(pred_test_h, on="id", how="inner")

    y_valid_h = valid_cal_h[f"default_within_{h}m"].values
    y_test_h = test_cal_h[f"default_within_{h}m"].values
    pd_raw_valid_h = valid_cal_h["pd_raw"].values
    pd_raw_test_h = test_cal_h["pd_raw"].values

    iso_h = IsotonicRegression(out_of_bounds="clip")
    iso_h.fit(pd_raw_valid_h, y_valid_h)
    cox_isotonic_models[h] = iso_h

    cox_calibration_rows.append({
        "horizon_months": h,
        "model": f"Cox_implied_PD_{h}m_raw",
        "n_valid": len(valid_cal_h),
        "n_test": len(test_cal_h),
        "valid_mean_pd": pd_raw_valid_h.mean(),
        "test_mean_pd": pd_raw_test_h.mean(),
        "valid_brier": brier_score_loss(y_valid_h, pd_raw_valid_h),
```

```
        "test_brier": brier_score_loss(y_test_h, pd_raw_test_h),
    })
    cox_calibration_rows.append({
        "horizon_months": h,
        "model": f"Cox_implied_PD_{h}m_calibrated",
        "n_valid": len(valid_cal_h),
        "n_test": len(test_cal_h),
        "valid_mean_pd": iso_h.transform(pd_raw_valid_h).mean(),
        "test_mean_pd": iso_h.transform(pd_raw_test_h).mean(),
        "valid_brier": brier_score_loss(y_valid_h, iso_h.
 ↪transform(pd_raw_valid_h)),
        "test_brier": brier_score_loss(y_test_h, iso_h.
 ↪transform(pd_raw_test_h)),
    })

# Backward-compatibility for downstream cells that reference this name directly.
cox_24_isotonic = cox_isotonic_models[LONG_HORIZON]

cox_calibration_summary = pd.DataFrame(cox_calibration_rows)
display(cox_calibration_summary)
```

```
   horizon_months                          model  n_valid  n_test  \
0              24         Cox_implied_PD_24m_raw   421095  707053
1              24  Cox_implied_PD_24m_calibrated   421095  707053
2              36         Cox_implied_PD_36m_raw   421042  536971
3              36  Cox_implied_PD_36m_calibrated   421042  536971
4              60         Cox_implied_PD_60m_raw    92520  116560
5              60  Cox_implied_PD_60m_calibrated    92520  116560

   valid_mean_pd  test_mean_pd  valid_brier  test_brier
0       0.136702      0.131277     0.111403    0.123877
1       0.133633      0.125351     0.110777    0.123167
2       0.204914      0.197635     0.137519    0.160093
3       0.174173      0.164467     0.136029    0.161111
4       0.352199      0.343181     0.217060    0.208869
5       0.363943      0.350576     0.215381    0.208141
```

2.5 7. Refit final models on train + validation

[19]:
```
# Combine train+valid for final fitting
tv_loans = pd.concat([train_loans, valid_loans], ignore_index=True)
tv_6 = pd.concat([train_6, valid_6], ignore_index=True)
tv_12 = pd.concat([train_12, valid_12], ignore_index=True)
tv_18 = pd.concat([train_18, valid_18], ignore_index=True)
tv_24 = pd.concat([train_24, valid_24], ignore_index=True)

# Final 6m logit
```

```
X_tv_6 = tv_6[model_numeric + model_categorical]
y_tv_6 = tv_6[f"default_within_{SIX_HORIZON}m"]

final_logit_6 = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=2000, C=0.5,␣
 ↪random_state=RANDOM_STATE)),
])
final_logit_6.fit(X_tv_6, y_tv_6)

# Final 18m logit
X_tv_18 = tv_18[model_numeric + model_categorical]
y_tv_18 = tv_18[f"default_within_{EIGHTEEN_HORIZON}m"]

final_logit_18 = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=2000, C=0.5,␣
 ↪random_state=RANDOM_STATE)),
])
final_logit_18.fit(X_tv_18, y_tv_18)

# Final 12m logit
X_tv_12 = tv_12[model_numeric + model_categorical]
y_tv_12 = tv_12[f"default_within_{SHORT_HORIZON}m"]

final_logit_12 = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=2000, C=0.5,␣
 ↪random_state=RANDOM_STATE)),
])
final_logit_12.fit(X_tv_12, y_tv_12)

# Final 24m logit
X_tv_24 = tv_24[model_numeric + model_categorical]
y_tv_24 = tv_24[f"default_within_{LONG_HORIZON}m"]

final_logit_24 = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=2000, C=0.5,␣
 ↪random_state=RANDOM_STATE)),
])
final_logit_24.fit(X_tv_24, y_tv_24)

# Final RF models for 6m and 18m
final_rf_raw_6 = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(
```

```
        n_estimators=200,
        max_depth=12,
        min_samples_leaf=50,
        max_features="sqrt",
        class_weight="balanced_subsample",
        max_samples=0.7,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )),
])
final_rf_6 = CalibratedClassifierCV(estimator=final_rf_raw_6, method="sigmoid",␣
 ↪cv=3)
final_rf_6.fit(X_tv_6, y_tv_6)

final_rf_raw_18 = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(
        n_estimators=200,
        max_depth=12,
        min_samples_leaf=50,
        max_features="sqrt",
        class_weight="balanced_subsample",
        max_samples=0.7,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )),
])
final_rf_18 = CalibratedClassifierCV(estimator=final_rf_raw_18,␣
 ↪method="sigmoid", cv=3)
final_rf_18.fit(X_tv_18, y_tv_18)

# Final RF models
final_rf_raw_12 = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(
        n_estimators=200,
        max_depth=12,
        min_samples_leaf=50,
        max_features="sqrt",
        class_weight="balanced_subsample",
        max_samples=0.7,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )),
])
final_rf_12 = CalibratedClassifierCV(estimator=final_rf_raw_12,␣
 ↪method="sigmoid", cv=3)
```

```
final_rf_12.fit(X_tv_12, y_tv_12)

final_rf_raw_24 = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(
        n_estimators=200,
        max_depth=12,
        min_samples_leaf=50,
        max_features="sqrt",
        class_weight="balanced_subsample",
        max_samples=0.7,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )),
])
final_rf_24 = CalibratedClassifierCV(estimator=final_rf_raw_24,␣
 ↪method="sigmoid", cv=3)
final_rf_24.fit(X_tv_24, y_tv_24)

# Final Cox
final_cox_train, final_cox_all, final_cox_features =␣
 ↪prepare_cox_design(tv_loans, df_model)
final_cph = CoxPHFitter(penalizer=0.1)
final_cph.fit(
    final_cox_train.drop(columns=["id"]),
    duration_col="duration_months",
    event_col="default_event",
    show_progress=False,
)
```

[19]: <lifelines.CoxPHFitter: fitted with 887440 total observations, 734375 right-censored observations>

2.6 8. Score

[20]:
```
pd_output = df_model[[
    "id",
    "grade",
    "issue_d",
    "loan_status",
    "loan_amnt",
    "int_rate",
    "annual_inc",
    "dti",
    "fico_avg",
    "term_num",
    "addr_state",
```

```
    "purpose",
    "home_ownership",
    "installment",
    "emp_length",
]].copy()

X_all = df_model[model_numeric + model_categorical]
pd_output["pd_6m_logit"] = final_logit_6.predict_proba(X_all)[:, 1]
pd_output["pd_12m_logit"] = final_logit_12.predict_proba(X_all)[:, 1]
pd_output["pd_18m_logit"] = final_logit_18.predict_proba(X_all)[:, 1]
pd_output["pd_24m_logit"] = final_logit_24.predict_proba(X_all)[:, 1]
pd_output["pd_6m_rf"] = final_rf_6.predict_proba(X_all)[:, 1]
pd_output["pd_12m_rf"] = final_rf_12.predict_proba(X_all)[:, 1]
pd_output["pd_18m_rf"] = final_rf_18.predict_proba(X_all)[:, 1]
pd_output["pd_24m_rf"] = final_rf_24.predict_proba(X_all)[:, 1]

# Cox implied PDs: use a direction-corrected and numerically stable mapping
cox_horizons = [SIX_HORIZON, EIGHTEEN_HORIZON, LONG_HORIZON, 36, 60]
lp_all = final_cph.
 ↪predict_log_partial_hazard(final_cox_all[final_cox_features]).values.ravel()
rel_risk = np.exp(np.clip(lp_all, -50, 50))
base_ch = final_cph.baseline_cumulative_hazard_.iloc[:, 0]
base_idx = base_ch.index.values.astype(float)
base_vals = base_ch.values.astype(float)

cox_all = pd.DataFrame({"id": final_cox_all["id"].values})
for h in cox_horizons:
    h0 = float(np.interp(float(h), base_idx, base_vals))
    surv_h = np.exp(-h0 * rel_risk)
    cox_all[f"pd_{h}m_cox_raw"] = 1 - surv_h

# Apply isotonic calibrators where available.
for h in [LONG_HORIZON, 36, 60]:
    raw_col = f"pd_{h}m_cox_raw"
    cal_col = f"pd_{h}m_cox"
    if "cox_isotonic_models" in globals() and h in cox_isotonic_models:
        cox_all[cal_col] = cox_isotonic_models[h].transform(cox_all[raw_col])
    else:
        cox_all[cal_col] = cox_all[raw_col]

pd_output = pd_output.merge(
    cox_all[[
        "id",
        "pd_6m_cox_raw",
        "pd_18m_cox_raw",
        "pd_24m_cox_raw",
        "pd_24m_cox",
```

```
        "pd_36m_cox_raw",
        "pd_36m_cox",
        "pd_60m_cox_raw",
        "pd_60m_cox",
    ]],
    on="id",
    how="left",
)

pd_output.loc[pd_output["term_num"] < 36, ["pd_36m_cox_raw", "pd_36m_cox"]] =␣
 ↪np.nan
pd_output.loc[pd_output["term_num"] < 60, ["pd_60m_cox_raw", "pd_60m_cox"]] =␣
 ↪np.nan

pd_output["pd_24m_chosen"] = pd_output[["pd_24m_logit", "pd_24m_rf",␣
 ↪"pd_24m_cox"]].median(axis=1)
#pd_output["chosen_longterm_model"] = chosen_longterm_model

display(pd_output.head())

pd_output.to_csv(OUTPUT_DIR / "pd_predictions.csv", index=False)
frequency_results.to_csv(OUTPUT_DIR / "frequency_model_metrics.csv",␣
 ↪index=False)
longterm_results.to_csv(OUTPUT_DIR / "longterm_model_comparison.csv",␣
 ↪index=False)
cox_summary.to_csv(OUTPUT_DIR / "cox_time_to_default_summary.csv", index=False)

print("Saved:")
print("-", OUTPUT_DIR / "pd_predictions.csv")
print("-", OUTPUT_DIR / "frequency_model_metrics.csv")
print("-", OUTPUT_DIR / "longterm_model_comparison.csv")
print("-", OUTPUT_DIR / "cox_time_to_default_summary.csv")
```

```
        id grade    issue_d loan_status  loan_amnt  annual_inc    dti  \
0  68407277     C 2015-12-01  Fully Paid     3600.0     55000.0   5.91
1  68355089     C 2015-12-01  Fully Paid    24700.0     65000.0  16.06
2  68341763     B 2015-12-01  Fully Paid    20000.0     63000.0  10.78
3  66310712     C 2015-12-01     Current    35000.0    110000.0  17.06
4  68476807     F 2015-12-01  Fully Paid    10400.0    104433.0  25.37

   fico_avg  term_num  pd_6m_logit  …  pd_6m_cox_raw  pd_18m_cox_raw  \
0     677.0      36.0     0.009495  …       0.013362        0.070474
1     717.0      36.0     0.035138  …       0.038372        0.191494
2     697.0      60.0     0.006474  …       0.013240        0.069849
3     787.0      60.0     0.005819  …       0.013285        0.070080
4     697.0      60.0     0.069483  …       0.022711        0.117330
```

```
   pd_24m_cox_raw  pd_24m_cox  pd_36m_cox_raw  pd_36m_cox  pd_60m_cox_raw  \
0        0.101131    0.080745        0.151817    0.101987             NaN
1        0.266641    0.332056        0.380563    0.407486             NaN
2        0.100250    0.074119        0.150533    0.096174        0.216771
3        0.100576    0.080745        0.151007    0.101235        0.217426
4        0.166465    0.181793        0.245123    0.233213        0.343694

   pd_60m_cox  pd_24m_chosen       chosen_longterm_model
0         NaN       0.078027  Median_24m_of_Logit_RF_Cox
1         NaN       0.228257  Median_24m_of_Logit_RF_Cox
2    0.168312       0.074119  Median_24m_of_Logit_RF_Cox
3    0.168312       0.080745  Median_24m_of_Logit_RF_Cox
4    0.343960       0.181793  Median_24m_of_Logit_RF_Cox

[5 rows x 27 columns]
          id grade    issue_d loan_status  loan_amnt  annual_inc    dti  \
0  68407277     C 2015-12-01  Fully Paid     3600.0     55000.0   5.91
1  68355089     C 2015-12-01  Fully Paid    24700.0     65000.0  16.06
2  68341763     B 2015-12-01  Fully Paid    20000.0     63000.0  10.78
3  66310712     C 2015-12-01     Current    35000.0    110000.0  17.06
4  68476807     F 2015-12-01  Fully Paid    10400.0    104433.0  25.37

   fico_avg  term_num  pd_6m_logit  …  pd_6m_cox_raw  pd_18m_cox_raw  \
0     677.0      36.0     0.009495  …       0.013362        0.070474
1     717.0      36.0     0.035138  …       0.038372        0.191494
2     697.0      60.0     0.006474  …       0.013240        0.069849
3     787.0      60.0     0.005819  …       0.013285        0.070080
4     697.0      60.0     0.069483  …       0.022711        0.117330

   pd_24m_cox_raw  pd_24m_cox  pd_36m_cox_raw  pd_36m_cox  pd_60m_cox_raw  \
0        0.101131    0.080745        0.151817    0.101987             NaN
1        0.266641    0.332056        0.380563    0.407486             NaN
2        0.100250    0.074119        0.150533    0.096174        0.216771
3        0.100576    0.080745        0.151007    0.101235        0.217426
4        0.166465    0.181793        0.245123    0.233213        0.343694

   pd_60m_cox  pd_24m_chosen       chosen_longterm_model
0         NaN       0.078027  Median_24m_of_Logit_RF_Cox
1         NaN       0.228257  Median_24m_of_Logit_RF_Cox
2    0.168312       0.074119  Median_24m_of_Logit_RF_Cox
3    0.168312       0.080745  Median_24m_of_Logit_RF_Cox
4    0.343960       0.181793  Median_24m_of_Logit_RF_Cox

[5 rows x 27 columns]

Saved:
- data/simplification_PD/pd_predictions.csv
- data/simplification_PD/frequency_model_metrics.csv
```

- data/simplification_PD/longterm_model_comparison.csv
- data/simplification_PD/cox_time_to_default_summary.csv